

SYSCLIN

MANUAL DE INFRAESTRUCTURA

Operación, despliegue y respaldo del sistema

Sistema SYSCLIN de gestión clínica

Versión 1.0

23/10/2026

Documento confidencial

CONTROL DEL DOCUMENTO

Campo	Valor
Documento	Manual de infraestructura
Sistema	Sysclin, gestión clínica (agendamiento, consultorio, ventas)
Destinatario	Soporte técnico: quien administra el servidor
Versión	1.0
Fecha de generación	23/10/2026
Revisión	Cada 6 meses, o después de un cambio de infraestructura
Confidencialidad	No contiene contraseñas, pero describe la infraestructura: no se publica ni se entrega a usuarios finales
Referencias	ISO/IEC 27002:2013 (A.12.1.1, procedimientos de operación documentados)

	Nombre	Firma	Fecha
Elaborado por			
Revisado por			
Aprobado por			

HISTORIAL DE REVISIONES

Versión	Fecha	Descripción del cambio
1.0	23/10/2026	Emisión inicial

ÍNDICE

1. Introducción....5
1.1 Qué encontrará en cada capítulo....5
1.2 Convenciones de este manual....5
2. Componentes del sistema (contenedores)....6
2.1 Archivos importantes....6
3. Configuración (.env)....8
3.1 Ver la configuración sin mostrar los secretos....8
3.2 Generar un valor aleatorio (contraseñas, SECRET_KEY)....8
3.3 Aplicar un cambio del .env....9
4. Manejo de contenedores y reactivación del sistema....10
4.1 Comandos de uso diario....10
4.2 Qué comando usar según el cambio....11
4.3 Comandos peligrosos....11
4.4 Reactivar el sistema....11
4.5 Modo de arranque de la aplicación....13
4.6 Tareas programadas....14
5. Base de datos: usuarios, contraseñas y migraciones....15
5.1 Ver los usuarios de la base....15
5.2 Consola SQL....15
5.3 Conectarse con DBeaver o pgAdmin....15
5.4 Aplicar una migración....15
5.5 Cambiar la contraseña de la aplicación (sysclin_app)....16
5.6 Cambiar la contraseña del administrador (postgres)....16
5.7 Comprobar los permisos de la aplicación....16
5.8 Comprobar que la contraseña por defecto no sirve....17
6. HTTPS, red y despliegue en el servidor....18
6.1 Protección de las comunicaciones....18
6.2 Exposición de red....18
6.3 HTTPS en desarrollo (certificado autofirmado)....19
6.4 Comprobar HTTPS....19

6.5 Desplegar en el servidor....19
6.6 Renovar el certificado (servidor)....20
7. Backups y restauración....21
7.1 Política....21
7.2 Cómo funciona....21
7.3 Custodia de la clave privada....22
7.4 Hacer un backup manual....22
7.5 Prueba de restauración....22
7.6 Restauración de emergencia....23
7.7 Si se pierde la clave privada....23
7.8 Generar un par de claves nuevo....24
8. Gestión de vulnerabilidades (dependencias)....25
8.1 Buscar vulnerabilidades conocidas....25
8.2 Ver qué versiones nuevas hay....25
8.3 Actualizar....25
8.4 Última auditoría (2026-10-11)....26
9. Solución de problemas....27
Anexo A. Scripts y archivos de configuración....29
A.1 dock/docker-compose.yml....29
A.2 dock/docker-compose.servidor.yml....31
A.3 dock/Dockerfile....31
A.4 .env.example....32
A.5 dock/backup/entrypoint.sh....32
A.6 dock/backup/backup.sh....33
A.7 dock/backup/restore_test.sh....34
A.8 dock/nginx/conf.d/default.conf....35
A.9 dock/nginx/hsts.conf....36
A.10 dock/nginx/generar_cert_local.sh....36
A.11 migrations/17_rol_sysclin_app.sql....36

1. Introducción

Este manual explica cómo **operar la infraestructura** de Sysclin: los contenedores, la configuración, la base de datos, el acceso por HTTPS, los backups y la solución de problemas. Está escrito para quien administra el servidor (soporte técnico); el administrador del consultorio no lo necesita para su trabajo diario.

No incluye las políticas de seguridad, que están en el **Manual de seguridad**, ni el uso de las pantallas, que explica el **Manual de usuario** (botón de Ayuda).

1.1 Qué encontrará en cada capítulo

Capítulo	Para qué sirve
Componentes del sistema	Qué contenedores hay, qué hace cada uno y cuáles son los archivos importantes
Configuración (.env)	Ver y cambiar la configuración sin exponer los secretos
Manejo de contenedores	Comandos de uso diario, cómo reactivar el sistema y las tareas programadas
Base de datos	Usuarios y contraseñas de la base, y cómo aplicar migraciones
HTTPS, red y despliegue	Certificados, exposición de puertos y cómo desplegar en el servidor
Backups y restauración	Copias cifradas, prueba de restauración y recuperación de emergencia
Gestión de vulnerabilidades	Buscar y aplicar actualizaciones de las dependencias
Solución de problemas	Síntomas frecuentes, su causa y qué hacer
Anexo A	El contenido vigente de los scripts y archivos de configuración

1.2 Convenciones de este manual

- Los comandos se ejecutan en **PowerShell**, desde la **carpeta raíz del proyecto** (syscomcli), salvo que se indique otra cosa. Los marcados como bash son para el servidor Linux.
- Donde aparece <algo>, se reemplaza por el valor real, sin los signos < >.
- El texto que empieza con **Nota:** es información a tener en cuenta; el que empieza con **Advertencia:**, una acción que puede causar pérdida de datos o corte del servicio.
- Las contraseñas y claves no aparecen en este manual: viven en el .env, que no se publica.

2. Componentes del sistema (contenedores)

Sysclin corre en **contenedores Docker** definidos en dock/docker-compose.yml. Cada contenedor cumple una sola función:

Contenedor	Función	Acceso desde la PC	Dentro de Docker
syscomcli_nginx	**Entrada oficial**: HTTPS; redirige HTTP a HTTPS	https://localhost (443) y http://localhost (80)	nginx
syscomcli_app	Aplicación web (Flask)	http://localhost:5005 (directo, solo desarrollo)	app:5000
syscomcli_bd	Base de datos PostgreSQL 17	localhost:5434	db:5432
syscomcli_backup	Backup automático cifrado cada 24 h	—	—
debugpy (en syscomcli_app)	Depurador remoto (solo desarrollo)	localhost:5680	app:5678

```
navegador ──HTTPS (443)──► nginx ──HTTP por la red interna──► app:5000 ──► db:5432
navegador ──HTTP (80)─────► nginx ──► redirige a HTTPS
backup ──pg_dump (cada 24 h)──► db:5432 ──► backups/*.dump.gpg (cifrado)
```

- Los usuarios entran por **https://localhost** (en el servidor, https://<dominio>).
- El puerto 5005 es un acceso directo a Flask sin cifrar; existe solo para desarrollo y en el servidor queda cerrado.
- **Red interna**: dock_network-tesis. La aplicación llega a la base por esa red (db:5432), no por el puerto 5434.
- **Datos**: volumen Docker tesis_syscomcli_data. Detener, recrear o actualizar los contenedores **no borra datos**.
- **Backups**: carpeta backups/ del proyecto.
- **Base de trabajo**: sysclin_bd. La base clinicain es legado y no se usa.
- Todos los contenedores tienen restart: always: si se caen o se reinicia Docker, vuelven a levantarse solos.

2.1 Archivos importantes

Archivo o carpeta	Contenido	¿Se sube a git?
.env	Contraseñas y configuración	**Nunca**
.env.example	Plantilla del .env sin valores	Sí
dock/docker-compose.yml	Contenedores de desarrollo	Sí

Archivo o carpeta	Contenido	¿Se sube a git?
dock/docker-compose.servidor.yml	Ajustes del servidor (se suma al anterior)	Sí
dock/Dockerfile	Imagen de la aplicación	Sí
dock/backup/	Scripts del backup y clave **pública**	Sí (la clave privada nunca)
dock/nginx/	Configuración de HTTPS	Sí, salvo certs/
dock/nginx/certs/	Certificado y su clave privada	**Nunca**
backups/	Backups cifrados	**Nunca**
migrations/	Cambios de la base, numerados	Sí

El contenido de cada script está en el Anexo.

3. Configuración (.env)

Toda la configuración sensible está en el archivo **.env** de la raíz del proyecto.

> **Advertencia:** El .env **nunca** se sube a git ni se comparte por chat o correo. Quien lo tiene, tiene acceso total a la base de datos.

Variable	Qué es	Desarrollo	Servidor
DB_HOST / DB_PORT	Dónde está la base, vista desde la app	db / 5432	igual
DB_NAME, DB_NAME_NUEVA	Base de trabajo	sysclin_bd	igual
DB_USER / DB_PASSWORD	Usuario de la **aplicación** (permisos mínimos)	sysclin_app / clave aleatoria	igual, otra clave
POSTGRES_USER / POSTGRES_PASSWORD	Usuario **administrador** de la base (backup, migraciones)	postgres / clave aleatoria	igual, otra clave
POSTGRES_DB	Base que se crea solo en una instalación nueva	clinicain	igual
SECRET_KEY	Firma de las sesiones. **Obligatoria**, mínimo 32 caracteres	aleatoria	aleatoria, **distinta**
SESSION_COOKIE_SECURE	Cookie de sesión solo por HTTPS	0	1
FLASK_DEBUG	Modo depuración	1	0
SMTP_*	Correo saliente (MFA, recuperación de contraseña, alertas)	cuenta Gmail	igual
ULTRAMSG_*	Recordatorios por WhatsApp	credenciales propias	igual

3.1 Ver la configuración sin mostrar los secretos

```
Get-Content .env | ForEach-Object { if ($_ -match 'PASSWORD|SECRET|TOKEN') { ($_ -split '=',2)[0] + '=***' } else { $_ } }
```

3.2 Generar un valor aleatorio (contraseñas, SECRET_KEY)

```
$rng = [Security.Cryptography.RandomNumberGenerator]::Create(); $b = New-Object byte[] 32; $rng.GetBytes($b); -join ($b | ForEach-Object { $_.ToString('x2') })
```

Genera 64 caracteres hexadecimales: solo letras y números, sin problemas de comillas ni símbolos.

3.3 Aplicar un cambio del .env

> **Nota:** docker restart **no** vuelve a leer el .env. Después de cambiarlo, siempre hay que **recrear** los contenedores:

```
docker compose -f dock/docker-compose.yml up -d
```

Si la variable la usa un solo servicio, alcanza con recrear ese: docker compose -f dock/docker-compose.yml up -d app.

> **Nota:** En docker-compose.yml, las variables del .env llegan a los contenedores solo con env_file:. Escribir ${VAR} dentro de environment: deja el valor vacío, porque Compose no lee el .env de la raíz para eso.

4. Manejo de contenedores y reactivación del sistema

En el servidor, **todos** los comandos de docker compose llevan los dos archivos: -f dock/docker-compose.yml -f dock/docker-compose.servidor.yml. En este capítulo se muestra la forma de desarrollo, con un solo archivo.

4.1 Comandos de uso diario

Tarea	Comando
Ver el estado	docker ps --filter name=syscomcli
Levantar todo	docker compose -f dock/docker-compose.yml up -d
Detener todo, sin borrar datos	docker compose -f dock/docker-compose.yml stop
Volver a arrancar lo detenido	docker compose -f dock/docker-compose.yml start
Reiniciar solo la app (sin cambios del .env)	docker restart syscomcli_app
Recrear un servicio (relee el .env)	docker compose -f dock/docker-compose.yml up -d <servicio> (app, db, backup, nginx)
Ver los logs de la app	docker logs syscomcli_app --tail 50
Seguir los logs en vivo	docker logs -f syscomcli_app (salir con Ctrl+C)
Logs de la última hora	docker logs syscomcli_app --since 1h
Comprobar que la app responde	curl.exe -sk -o NUL -w "%{http_code}" https://localhost/login → 200
Abrir una terminal dentro de un contenedor	docker exec -it syscomcli_app sh (salir con exit)
Uso de CPU y memoria	docker stats --no-stream --filter name=syscomcli
Espacio en disco de Docker	docker system df

Estado esperado: 4 contenedores Up (syscomcli_nginx, syscomcli_app, syscomcli_bd, syscomcli_backup); la base, además, (healthy).

4.2 Qué comando usar según el cambio

Se cambió…	Hay que…	Comando
Código Python o plantillas (desarrollo)	Nada: se recarga solo	—
Código Python o plantillas (servidor)	Reiniciar la app	docker restart syscomcli_app
El .env	Recrear	docker compose -f dock/docker-compose.yml up -d
docker-compose.yml	Recrear	docker compose -f dock/docker-compose.yml up -d
requirements.txt o Dockerfile	Reconstruir la imagen y recrear	docker compose -f dock/docker-compose.yml build app y luego up -d app
Configuración de nginx	Validar y recargar	docker exec syscomcli_nginx nginx -t y luego docker exec syscomcli_nginx nginx -s reload
Scripts del backup	Reiniciar el backup	docker restart syscomcli_backup

Nota: Recrear o reconstruir la app corta el servicio entre 30 y 60 segundos y **cierra las sesiones** de todos los usuarios. Avisar antes o hacerlo fuera del horario de atención.

4.3 Comandos peligrosos

Comando	Qué hace
docker compose -f dock/docker-compose.yml down	Borra los contenedores y la red. Los datos **no** se pierden (están en el volumen); se recupera con up -d
docker compose ... down -v	**Borra el volumen de la base: se pierden todos los datos.** No usarlo nunca en producción
docker volume rm tesis_syscomcli_data	**Ídem: borra la base**
docker system prune -a --volumes	**Borra todo lo que no esté corriendo, incluidos volúmenes**

Advertencia: Antes de cualquiera de estos, hacer un backup manual (capítulo Backups) y comprobar que el archivo se creó.

4.4 Reactivar el sistema

4.4.1 A. Después de reiniciar o apagar la PC o el servidor

1. Abrir **Docker Desktop** (en Windows) y esperar a que diga *Engine running*. En Linux, Docker arranca solo con el sistema.

2. Los contenedores vuelven a levantarse **solos** (restart: always). Comprobar con docker ps --filter name=syscomcli.
3. Si alguno no aparece, levantar todo: docker compose -f dock/docker-compose.yml up -d.
4. Entrar a https://localhost e iniciar sesión.
5. El backup hace una copia apenas arranca: confirmarlo con docker logs syscomcli_backup --tail 5.

4.4.2 B. La página no carga o muestra "502 Bad Gateway"

1. docker ps -a --filter name=syscomcli: ver qué contenedor no está Up o se reinicia una y otra vez (Restarting).
2. Leer su log: docker logs <contenedor> --tail 50. El error suele estar en las últimas líneas.
3. Si es la app y el error es de configuración (por ejemplo SECRET_KEY ausente o password authentication failed), corregir el .env y recrear (capítulo Solución de problemas).
4. Si no hay un error claro, reiniciar ese contenedor: docker restart <contenedor>.
5. Si sigue igual, recrearlo: docker compose -f dock/docker-compose.yml up -d --force-recreate <servicio>.

4.4.3 C. La base de datos no arranca

1. docker logs syscomcli_bd --tail 50.
2. Si dice No space left on device: liberar espacio en disco (docker system df; borrar imágenes viejas con docker image prune, **nunca** volúmenes).
3. Si el log indica datos dañados y no se recupera con un reinicio, hacer la **restauración de emergencia** (capítulo Backups).

4.4.4 D. Las tareas automáticas no se ejecutan (recordatorios, turnos)

Ver el apartado "Tareas programadas" más abajo. En general alcanza con docker restart syscomcli_app.

4.4.5 E. Instalación nueva desde cero (otra PC u otro servidor)

1. Instalar Docker (Docker Desktop en Windows; Docker Engine en Linux) y copiar el proyecto.
2. Crear el .env a partir de .env.example y generar **todas** las claves nuevas (capítulo Configuración). En el servidor: SESSION_COOKIE_SECURE=1 y FLASK_DEBUG=0.

3. Generar el certificado de HTTPS (capítulo HTTPS): autofirmado en desarrollo, Let's Encrypt en el servidor.
4. Comprobar que exista dock/backup/keys/backup_pub.asc (la clave pública del backup).
5. Levantar solo la base: docker compose -f dock/docker-compose.yml up -d db. Al ser un volumen nuevo, crea el usuario postgres con la contraseña del .env.
6. Crear la base de trabajo y el rol de la aplicación:

```
docker exec syscomcli_bd psql -U postgres -c "CREATE DATABASE sysclin_bd"
$appPwd = (Select-String -Path .env -Pattern '^DB_PASSWORD=(.*)$').Matches[0].Groups[1].Value
Get-Content migrations\17_rol_sysclin_app.sql -Raw | docker exec -i syscomcli_bd psql -U postgres -d sysclin_bd -v
ON_ERROR_STOP=1 -v "app_password=$appPwd" -q
```

7. Cargar los datos desde el último backup con la **restauración de emergencia** (capítulo Backups, pasos 4 y siguientes).
8. Volver a ejecutar el script 17 de migrations/ (mismo comando del paso 6), para que los permisos de sysclin_app queden completos.
9. Levantar todo: docker compose -f dock/docker-compose.yml up -d (en el servidor, con los dos archivos).
10. Hacer el **chequeo periódico** completo (Manual de seguridad, capítulo «Auditoría y chequeo periódico»).

4.5 Modo de arranque de la aplicación

El contenedor syscomcli_app arranca de una de dos formas, según FLASK_DEBUG del .env:

FLASK_DEBUG	Arranca con	Para
1	Servidor de desarrollo de Flask + debugpy (puerto 5678), con recarga automática al guardar código	Desarrollo
0	**gunicorn**: 1 proceso con 8 hilos y un límite de 120 s por petición	Servidor

En el servidor se usa **un solo proceso a propósito**: las tareas programadas arrancan una vez por proceso y, con más procesos, los recordatorios se enviarían repetidos.

Ver en qué modo está corriendo:

```
docker exec syscomcli_app sh -c "cat /proc/1/cmdline | tr '\0' ' '"
```

Se espera python -m debugpy ... run.py en desarrollo y gunicorn --bind 0.0.0.0:5000 --workers 1 --threads 8 ... en el servidor.

4.6 Tareas programadas

Los recordatorios por WhatsApp, la extensión de turnos de agenda y la limpieza de sesiones corren dentro de la app, en **un único proceso**. Lo garantiza un candado en PostgreSQL (advisory lock 7420001): el proceso que lo toma ejecuta las tareas; cualquier otro lo avisa en el log y no las arranca.

Ver quién ejecuta las tareas (debe haber exactamente **una** fila):

```
docker exec syscomcli_bd psql -U postgres -d sysclin_bd -Atc "SELECT l.pid, a.usename, host(a.client_addr) FROM pg_locks l JOIN pg_stat_activity a ON a.pid=l.pid WHERE l.locktype='advisory' AND l.objid=7420001"
```

- **Ninguna fila**: ningún proceso ejecuta las tareas. Reiniciar la app: docker restart syscomcli_app.
- Ver los arranques en el log: docker logs syscomcli_app --since 1h 2>&1 | Select-String 'Scheduler iniciado|NO iniciadas'.

5. Base de datos: usuarios, contraseñas y migraciones

5.1 Ver los usuarios de la base

```
docker exec syscomcli_bd psql -U postgres -d sysclin_bd -c "\du"
```

Esperado: postgres con Superuser, Create role, Create DB, Replication, Bypass RLS, y sysclin_app sin atributos. El rol y sus permisos los crea migrations/17_rol_sysclin_app.sql; las tablas, vistas y funciones que se creen después reciben los mismos permisos automáticamente.

5.2 Consola SQL

```
# Como administrador (todos los permisos)
docker exec -it syscomcli_bd psql -U postgres -d sysclin_bd

# Como la aplicación (permisos mínimos): para comprobar qué puede hacer la app
docker exec -it syscomcli_bd psql -U sysclin_app -d sysclin_bd
```

Estas consolas no piden contraseña porque se conectan desde dentro del contenedor. Desde fuera (DBeaver, otra PC) **siempre** se pide. Para salir: \q.

5.3 Conectarse con DBeaver o pgAdmin

Campo	Valor
Host / Puerto	localhost / 5434
Base	sysclin_bd
Usuario / Contraseña	postgres / la de POSTGRES_PASSWORD del .env

En el servidor el puerto de la base no se publica a internet: conectarse por **túnel SSH** (pestaña "SSH" de la conexión en DBeaver, o ssh -L 5434:127.0.0.1:5434 <usuario>@<servidor>).

5.4 Aplicar una migración

Las migraciones están en migrations/, numeradas, y se aplican en orden, como administrador. **Antes, hacer un backup manual** (capítulo Backups).

```
Get-Content migrations\<NNN_nombre>.sql -Raw | docker exec -i syscomcli_bd psql -U postgres -d sysclin_bd -v
ON_ERROR_STOP=1
```

ON_ERROR_STOP=1 frena al primer error, así no queda una migración aplicada a medias sin aviso.

5.5 Cambiar la contraseña de la aplicación (sysclin_app)

Cada 6 meses, o enseguida si se sospecha que se filtró. Hacerlo fuera del horario de atención: entre los pasos 2 y 3 la app no puede conectarse.

1. Generar un valor nuevo (capítulo Configuración) y ponerlo en DB_PASSWORD del .env.
2. Aplicarlo en la base:

```
$appPwd = (Select-String -Path .env -Pattern '^DB_PASSWORD=(.*)$').Matches[0].Groups[1].Value
Get-Content migrations\17_rol_sysclin_app.sql -Raw | docker exec -i syscomcli_bd psql -U postgres -d sysclin_bd -v
ON_ERROR_STOP=1 -v "app_password=$appPwd" -q
```

3. Recrear la app: docker compose -f dock/docker-compose.yml up -d app.
4. Comprobar que responde y que se puede iniciar sesión.

5.6 Cambiar la contraseña del administrador (postgres)

1. Generar un valor nuevo y ponerlo en POSTGRES_PASSWORD del .env.
2. Aplicarlo en la base:

```
$admPwd = (Select-String -Path .env -Pattern '^POSTGRES_PASSWORD=(.*)$').Matches[0].Groups[1].Value
docker exec syscomcli_bd psql -U postgres -qc "ALTER ROLE postgres PASSWORD '$admPwd'"
```

3. Recrear el backup para que tome la contraseña nueva: docker compose -f dock/docker-compose.yml up -d backup.
4. Hacer un backup manual para confirmar que funciona.
5. Actualizar la contraseña guardada en DBeaver o pgAdmin.

> **Nota:** Con una base ya creada, cambiar solo el .env **no** cambia la contraseña: siempre hace falta el paso 2. Solo en una instalación nueva (volumen vacío) el contenedor crea postgres directamente con la contraseña del .env.

5.7 Comprobar los permisos de la aplicación

```
# Deben dar ERROR (la aplicación no puede hacerlo)
docker exec syscomcli_bd psql -U sysclin_app -d sysclin_bd -c "DROP TABLE usuarios"
docker exec syscomcli_bd psql -U sysclin_app -d sysclin_bd -c "CREATE TABLE zz_prueba (id int)"
docker exec syscomcli_bd psql -U sysclin_app -d sysclin_bd -c "TRUNCATE auditoria_sistema"

# Debe funcionar (la aplicación sí puede leer)
docker exec syscomcli_bd psql -U sysclin_app -d sysclin_bd -Atc "SELECT current_user, count(*) FROM usuarios
GROUP BY 1"
```

Comando	Resultado esperado
DROP TABLE	ERROR: must be owner of table usuarios
CREATE TABLE	ERROR: permission denied for schema public
TRUNCATE	ERROR: permission denied for table auditoria_sistema
SELECT	sysclin_app|<cantidad de usuarios>

5.8 Comprobar que la contraseña por defecto no sirve

```
docker run --rm --network dock_network-tesis -e PGPASSWORD=postgres postgres:17 psql -h db -U postgres -d sysclin_bd -c "SELECT 1"
```

Esperado: FATAL: password authentication failed for user "postgres".

6. HTTPS, red y despliegue en el servidor

6.1 Protección de las comunicaciones

- Todo acceso de los usuarios es por **HTTPS** (nginx), con TLS 1.2 o 1.3; las versiones anteriores se rechazan.
- HTTP redirige siempre a HTTPS. En el servidor se activa **HSTS** por 1 año.
- Encabezados de seguridad: X-Frame-Options: SAMEORIGIN (el sistema no se puede abrir dentro de otro sitio), X-Content-Type-Options: nosniff, Referrer-Policy: strict-origin-when-cross-origin.
- Certificado: Let's Encrypt en el servidor, renovado cada 60 días; autofirmado solo en desarrollo.
- La configuración de nginx es **la misma** en desarrollo y en el servidor; solo cambian el certificado, el HSTS y los puertos cerrados.

6.2 Exposición de red

Puerto	Servicio	Desarrollo	Servidor
443 / 80	nginx (HTTPS / redirección)	Abierto	Abierto: **única entrada**
5005	Aplicación directa, sin cifrar	Abierto	Solo 127.0.0.1
5434	Base de datos	Abierto	Solo 127.0.0.1; administración por túnel SSH
5680	Depurador (debugpy)	Abierto	Solo 127.0.0.1; además no corre en el servidor

Nota: No alcanza con cerrar puertos con ufw en el servidor: Docker lo saltea y publica los puertos igual. Por eso el cierre se hace en docker-compose.servidor.yml, y en el servidor el sistema se levanta **siempre** con los dos archivos.

Ver qué puertos va a publicar cada configuración (no levanta nada):

```
docker compose -f dock/docker-compose.yml -f dock/docker-compose.servidor.yml config --format json | ConvertFrom-Json | ForEach-Object { $_.services.PSObject.Properties | ForEach-Object { $n=$_.Name; $_.Value.ports | ForEach-Object { "{0,-7} {1}:{2} -> {3}" -f $n, $_.host_ip, $_.published, $_.target } } }
```

Esperado en el servidor: 127.0.0.1 delante de 5005, 5434 y 5680; 80 y 443 sin IP.

6.3 HTTPS en desarrollo (certificado autofirmado)

```
# Generar el certificado (una vez; vale 825 días)
docker run --rm -v "${PWD}\dock\nginx:/nginx" postgres:17 sh /nginx/generar_cert_local.sh

# Levantar o recrear nginx
docker compose -f dock/docker-compose.yml up -d nginx
```

El navegador avisa que el certificado no es de confianza: es lo esperado con uno autofirmado. Se acepta una vez ("Configuración avanzada → Continuar").

6.4 Comprobar HTTPS

```
# Validar la configuración de nginx
docker exec syscomcli_nginx nginx -t

# HTTP redirige a HTTPS (esperado: 301 -> https://localhost/login)
curl.exe -s -o NUL -w "%{http_code} -> %{redirect_url}`n" http://localhost/login

# Encabezados de seguridad
curl.exe -sk -D - -o NUL https://localhost/login | Select-String -Pattern '^HTTP|X-Frame|X-Content|Referrer|Strict-Transport'

# Versión de TLS (esperado: TLSv1.3)
docker run --rm --network dock_network-tesis postgres:17 sh -c "echo | openssl s_client -connect nginx:443 -brief 2>&1 | grep -E 'Protocol|Ciphersuite'"

# TLS 1.1 rechazado (esperado: 000 = sin conexión)
curl.exe -sk --tlsv1.1 --tls-max 1.1 -o NUL -w "%{http_code}`n" https://localhost/login
```

Logs de acceso de nginx: `docker logs syscomcli_nginx --tail 50`.

6.5 Desplegar en el servidor

Requisitos: servidor Linux con Docker; un **dominio** que apunte a su IP (Let's Encrypt no emite certificados para IP); puertos 80 y 443 abiertos en el firewall del proveedor.

1. Copiar el proyecto y crear su `.env` desde `.env.example`, con **todas las claves nuevas**, `SESSION_COOKIE_SECURE=1` y `FLASK_DEBUG=0`.
2. Obtener el certificado de Let's Encrypt. Primero se levanta nginx con un certificado autofirmado provisorio (comando de desarrollo de arriba) para que responda en el puerto 80:

```
docker compose -f dock/docker-compose.yml -f dock/docker-compose.servidor.yml up -d
docker run --rm -v "$PWD/dock/nginx/acme:/var/www/acme" -v "/etc/letsencrypt:/etc/letsencrypt" \
  certbot/certbot certonly --webroot -w /var/www/acme -d <dominio> --email <correo> --agree-tos --no-eff-email
cp -L /etc/letsencrypt/live/<dominio>/fullchain.pem dock/nginx/certs/fullchain.pem
cp -L /etc/letsencrypt/live/<dominio>/privkey.pem dock/nginx/certs/privkey.pem
docker exec syscomcli_nginx nginx -s reload
```

3. **Siempre** levantar con los dos archivos:

```
docker compose -f dock/docker-compose.yml -f dock/docker-compose.servidor.yml up -d
```

4. Comprobar HTTPS con el dominio (debe aparecer Strict-Transport-Security), que la cookie tenga Secure y que los puertos 5005, 5434 y 5680 **no** respondan desde otra máquina.

6.6 Renovar el certificado (servidor)

Los certificados de Let's Encrypt duran 90 días. Renovarlos cada 60; conviene programarlo con cron:

```
docker run --rm -v "$PWD/dock/nginx/acme:/var/www/acme" -v "/etc/letsencrypt:/etc/letsencrypt" certbot/certbot renew
cp -L /etc/letsencrypt/live/<dominio>/fullchain.pem dock/nginx/certs/fullchain.pem
cp -L /etc/letsencrypt/live/<dominio>/privkey.pem dock/nginx/certs/privkey.pem
docker exec syscomcli_nginx nginx -s reload
```

Ver la fecha de vencimiento del certificado instalado:

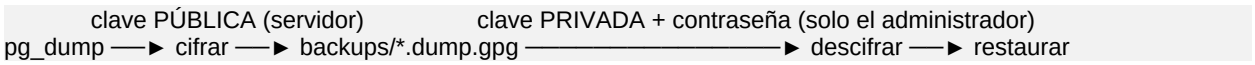
```
docker run --rm -v "${PWD}\dock\nginx\certs:/c:ro" postgres:17 openssl x509 -in /c/fullchain.pem -noout -subject -enddate
```

7. Backups y restauración

7.1 Política

Aspecto	Política
Frecuencia	Automático cada 24 h (y uno apenas arranca el contenedor). Manual antes de cada migración, actualización o despliegue
Retención	14 días; los más viejos se borran solos
Cifrado	GPG asimétrico. El servidor tiene solo la clave pública: **puede cifrar, no descifrar**. Nunca se genera un backup sin cifrar
Custodia	Clave privada y su contraseña en poder del administrador, guardadas por separado, fuera del servidor
Prueba de restauración	**Mensual**, y antes de cada entrega o despliegue. Un backup que no se probó no se da por válido
Copia fuera del equipo	Recomendada, semanal. Al estar cifrados, pueden guardarse en una nube sin exponer datos
Objetivo de recuperación	Pérdida máxima de 24 h de datos

7.2 Cómo funciona



- Archivo: backups/sysclin_bd_AAAAMMDD_HHMMSS.dump.gpg.
- Clave pública: dock/backup/keys/backup_pub.asc. Aunque alguien se lleve el servidor, los backups son ilegibles.
- Si falta la clave pública, el backup **no se hace**. El dump sin cifrar existe solo un instante dentro del contenedor y se borra siempre, salga bien o mal.
- Los scripts son dock/backup/entrypoint.sh (ciclo cada 24 h) y dock/backup/backup.sh (la copia); ver el Anexo.

7.3 Custodia de la clave privada

Elemento	Dónde
Clave pública	dock/backup/keys/backup_pub.asc. Se puede compartir
Clave privada	backup_privada.asc, en USB o gestor de contraseñas del administrador. **Nunca** en el servidor ni en el repositorio
Contraseña de la clave privada	Gestor de contraseñas, **separada** del archivo
Huella de la clave vigente	5CBD D0C3 C3A5 7281 CEE5 AF1B C0CE EFF6 A6B5 D972

Advertencia: Sin la clave privada y su contraseña, ningún backup se puede recuperar.

Conviene tener dos copias de la clave privada en lugares distintos.

En los comandos siguientes, <ruta_clave_privada> es la ubicación del archivo (por ejemplo E:\backup_privada.asc en un USB).

Ver la huella de la clave pública instalada:

```
docker run --rm -v "${PWD}\dock\backup\keys\backup_pub.asc:/k/pub.asc:ro" postgres:17 bash -c "mkdir -m 700 /tmp/g && GNUPGHOME=/tmp/g gpg --show-keys --with-fingerprint /k/pub.asc"
```

7.4 Hacer un backup manual

```
docker exec syscomcli_backup sh /scripts/backup.sh
```

Termina con Backup creado: sysclin_bd_<fecha>.dump.gpg (<tamaño>) y la lista de backups.

```
# Listar los backups
Get-ChildItem backups | Select-Object Name, Length, LastWriteTime

# Ver los últimos backups automáticos y si hubo errores
docker logs syscomcli_backup --tail 20

# Copia fuera del equipo (por ejemplo, a un USB en E:)
Copy-Item backups\*.dump.gpg E:\sysclin_backups\
```

7.5 Prueba de restauración

Una vez por mes y antes de cada entrega o despliegue. **La base real no se toca.**

```
# El backup más reciente
docker run --rm -it --network dock_network-tesis --env-file .env -v "${PWD}\backups:/backups:ro" -v "${PWD}\dock\backup:/scripts:ro" -v "<ruta_clave_privada>:/clave/backup_privada.asc:ro" postgres:17 sh /scripts/restore_test.sh

# Un backup en particular: agregar su nombre al final
# ... postgres:17 sh /scripts/restore_test.sh sysclin_bd_<fecha>.dump.gpg
```

El script (dock/backup/restore_test.sh):

1. pide la contraseña de la clave privada;
2. descifra el backup;
3. lo restaura en una base temporal (sysclin_restore_test);
4. compara la cantidad de registros de usuarios, pacientes, citas, consultas, facturas y auditoria_sistema contra la base real;
5. borra la base temporal y el archivo descifrado.

Resultado esperado: una tabla con las cantidades y RESTAURACION OK. Que "restaurada" sea menor que "actual" es normal: corresponde a lo cargado después del backup. Si dice RESTAURACION FALLO: <motivo>, el motivo indica el problema (no se encontró el backup, falta la clave, contraseña incorrecta o archivo dañado).

7.6 Restauración de emergencia

> **Advertencia:** Solo si la base se dañó o se perdió. **Reemplaza el contenido de sysclin_bd** por el del backup elegido.

1. Si la base todavía responde, hacer antes un backup del estado actual.
2. Elegir el backup (listado de arriba).
3. Detener la aplicación y el backup; la base queda corriendo:

```
docker compose -f dock/docker-compose.yml stop app backup
```

4. Descifrar y restaurar (pide la contraseña de la clave):

```
docker run --rm -it --network dock_network-tesis --env-file .env -v "${PWD}\backups:/backups:ro" -v
"<ruta_clave_privada>:/clave/k.asc:ro" postgres:17 bash -c 'mkdir -m 700 /tmp/g && export GNUPGHOME=/tmp/g
&& gpg --batch --quiet --import /clave/k.asc && gpg --pinentry-mode loopback --output /tmp/r.dump --decrypt
/backups/<archivo>.dump.gpg && PGPASSWORD=$POSTGRES_PASSWORD pg_restore -h db -U
$POSTGRES_USER -d $DB_NAME --clean --if-exists /tmp/r.dump'
```

5. Volver a levantar:

```
docker compose -f dock/docker-compose.yml start app backup
```

6. Comprobar que se puede iniciar sesión y que los datos están.

Los permisos de sysclin_app sobre las tablas vienen incluidos en el backup.

7.7 Si se pierde la clave privada

- Los backups ya cifrados con esa clave **no se pueden recuperar**.

- Generar enseguida un par de claves nuevo (abajo), hacer un backup manual y una prueba de restauración.

7.8 Generar un par de claves nuevo

Solo en una instalación nueva, o si se perdió o se comprometió la clave privada. Pide la contraseña de la clave dos veces.

```
$dir = "$env:USERPROFILE\sysclin_clave_backup"
New-Item -ItemType Directory -Force $dir | Out-Null
docker run --rm -it -v "${dir}:/clave" postgres:17 bash -c "mkdir -m 700 /tmp/g && export GNUPGHOME=/tmp/g &&
gpg --pinentry-mode loopback --quick-gen-key 'Sysclin Backup <backup@sysclin.local>' default default never && gpg
--armor --export 'Sysclin Backup' > /clave/backup_pub.asc && gpg --pinentry-mode loopback --armor --export-secret-
keys 'Sysclin Backup' > /clave/backup_privada.asc && gpg --list-keys"

# Instalar la pública y reiniciar el backup
Copy-Item "$dir\backup_pub.asc" dock\backup\keys\backup_pub.asc
docker restart syscomcli_backup
```

1. Pasar backup_privada.asc a su lugar de custodia y **borrarla de la PC**.
2. Anotar la huella nueva en la tabla de custodia.
3. Hacer una prueba de restauración.

Conservar la clave privada anterior mientras existan backups cifrados con ella (14 días).

8. Gestión de vulnerabilidades (dependencias)

- Las librerías de Python están en requirements.txt, cada una con su **versión exacta**. Solo se instalan las que el sistema usa.
- La revisión se hace **cada mes**, antes de cada despliegue y cuando se agrega una librería.
- Las imágenes base (python:3.12-slim, postgres:17, nginx:1.27-alpine) se actualizan junto con las dependencias.

8.1 Buscar vulnerabilidades conocidas

Corre en un contenedor temporal, sin tocar la app:

```
docker run --rm -v "${PWD}\requirements.txt:/r/requirements.txt:ro" python:3.12-slim sh -c "pip install -q --root-user-action=ignore --disable-pip-version-check pip-audit && pip-audit -r /r/requirements.txt --progress-spinner off"
```

Esperado: No known vulnerabilities found. Si aparece alguna, la columna Fix Versions indica la versión que la corrige.

8.2 Ver qué versiones nuevas hay

```
docker run --rm -v "${PWD}\requirements.txt:/r/requirements.txt:ro" python:3.12-slim sh -c "pip install -q --root-user-action=ignore --disable-pip-version-check -r /r/requirements.txt && pip list --outdated"
```

8.3 Actualizar

1. Cambiar la versión en requirements.txt.
2. Volver a correr el paso 1.
3. Reconstruir la imagen y recrear la app (capítulo Contenedores). Corta el servicio unos 30 a 60 segundos.
4. Probar: inicio de sesión, una pantalla de cada módulo, un PDF de factura y uno de reporte, y este manual en PDF.

Si una actualización de versión mayor (por ejemplo, de 4.x a 5.x) rompe algo, volver a la versión anterior y reconstruir. Si una vulnerabilidad **no** tiene corrección, documentarla como riesgo aceptado con su mitigación.

Antes de quitar una librería, confirmar que nada la importe:

```
Get-ChildItem app, run.py -Recurse -Include *.py | Select-String -Pattern "^\s*(import|from)\s+<modulo>\b"
```

8.4 Última auditoría (2026-10-11)

Indicador	Antes	Después
Paquetes en requirements.txt	53	30
Paquetes instalados, con dependencias indirectas	81	64
Paquetes vulnerables	11	**0**
Vulnerabilidades conocidas	85	**0**

Se eliminaron 23 paquetes sin uso y los 30 restantes se actualizaron y se fijaron con versión exacta.

9. Solución de problemas

Síntoma	Causa	Solución
Cambié el .env y no pasa nada	docker restart no relee el .env	docker compose -f dock/docker-compose.yml up -d
La app no arranca; el log dice SECRET_KEY ausente o con menos de 32 caracteres	Falta la SECRET_KEY o es corta	Generar una, ponerla en el .env y recrear
La app no conecta: password authentication failed for user "sysclin_app"	La contraseña del .env no coincide con la de la base	Repetir "Cambiar la contraseña de la aplicación" (capítulo Base de datos)
El backup falla: password authentication failed for user "postgres"	POSTGRES_PASSWORD del .env no coincide con la base	Corregir el .env o repetir el cambio de contraseña del administrador, y recrear el backup
El backup dice falta la clave publica	No existe dock/backup/keys/backup_pub.asc	Restaurarlo desde git o generar un par nuevo
La prueba de restauración dice no se pudo descifrar	Contraseña incorrecta, o el backup se cifró con otra clave	Revisar la contraseña; usar la clave privada que corresponde a la fecha del backup
DBeaver no conecta después de cambiar la contraseña	Quedó guardada la contraseña vieja	Actualizarla en la conexión
Un .sh falla con \r: not found	El archivo tiene saltos de línea de Windows (CRLF)	Guardarlo con saltos LF (en VS Code: esquina inferior derecha, "CRLF" → "LF")
En Git Bash, una ruta /var/... aparece como C:/Program Files/Git/var/...	Git Bash convierte las rutas	Usar PowerShell, o anteponer MSYS_NO_PATHCONV=1
postgres/postgres parece funcionar con docker exec ... psql -h 127.0.0.1	Dentro del contenedor no se pide contraseña	Es esperado; la prueba válida es la del capítulo Base de datos
Un usuario ve "Cuenta bloqueada. Contacte al administrador"	Su cuenta llegó al nivel 3	Desbloquear desde Auditoría → Sesiones y usuarios, o desde Usuarios. Revisar antes en Auditoría si los intentos fueron suyos
Todos los de una oficina ven "Demasiados intentos fallidos desde esta IP"	Comparten la IP pública y entre todos sumaron 10 fallos en 15 min	Esperar 15 minutos; revisar el RATE_LIMIT en Auditoría
Un usuario no recibe el correo de recuperación	Pidió más de 3 en una hora, o el usuario y el correo no coinciden	Esperar 60 min, o revisar el RATE_LIMIT. El mensaje en pantalla es siempre el mismo, a propósito
Los recordatorios de WhatsApp llegan repetidos	Más de un proceso tiene las tareas (no debería pasar)	Consulta del candado (capítulo Contenedores); debe haber una sola fila

Síntoma	Causa	Solución
No se mandan recordatorios ni se extienden los turnos	Ningún proceso tiene el candado	docker restart syscomcli_app y revisar el log (NO iniciadas / No se pudo verificar el candado)
Un PDF (ficha, factura) da error 500 y el log dice No such file or directory: ... .html	Falta la plantilla HTML del documento	Restaurarla desde git: app/rutas/clinico/movimientos/ficha/templates/FICHA.html y app/rutas/ventas/movimientos/factura/templates/FE.html
El único administrador quedó bloqueado	Nivel 3, y nadie más puede desbloquearlo	Desbloqueo de emergencia por consola (Manual de seguridad, capítulo «Seguridad de la aplicación»)
El navegador avisa "la conexión no es privada" en https://localhost	Certificado de desarrollo autofirmado	Esperado en local; aceptar la advertencia
nginx no arranca: cannot load certificate	Faltan fullchain.pem o privkey.pem en dock/nginx/certs/	Generarlos o copiarlos de Let's Encrypt (capítulo HTTPS)
https://... da 502 Bad Gateway	nginx funciona, pero la app está caída o reiniciando	docker logs syscomcli_app --tail 50; ver "Reactivar el sistema"
Error de CSRF al iniciar sesión por HTTPS	Se cambió proxy_set_header Host en nginx	Debe quedar Host $http_host
En el servidor, el 5005 o la base responden desde otra PC	Se levantó sin docker-compose.servidor.yml	Levantar con los dos archivos
Una variable ${VAR} queda vacía en docker-compose.yml	Compose no lee el .env de la raíz para ${VAR}	Usar env_file:, no ${VAR} en environment:
Docker Desktop no arranca o los contenedores no aparecen	El motor de Docker está detenido	Abrir Docker Desktop, esperar *Engine running* y ejecutar up -d
Un contenedor aparece como Restarting todo el tiempo	Falla al arrancar y Docker lo reintenta	Leer su log (docker logs <contenedor> --tail 50) y corregir la causa

Anexo A. Scripts y archivos de configuración

Contenido vigente de cada archivo al 23/10/2026, tomado directamente del sistema. Las rutas son relativas a la carpeta raíz del proyecto (syscomcli). Las contraseñas no aparecen: viven en el .env, que no forma parte de este anexo.

Archivo	Para qué sirve
dock/docker-compose.yml	Definición de los contenedores (desarrollo)
dock/docker-compose.servidor.yml	Ajustes del servidor: puertos cerrados y HSTS
dock/Dockerfile	Imagen de la aplicación y modo de arranque
.env.example	Plantilla del .env, sin valores
dock/backup/entrypoint.sh	Servicio de backup: ciclo cada 24 h
dock/backup/backup.sh	Backup cifrado con GPG y rotación
dock/backup/restore_test.sh	Prueba de restauración
dock/nginx/conf.d/default.conf	Proxy HTTPS (nginx)
dock/nginx/hsts.conf	HSTS (solo servidor)
dock/nginx/generar_cert_local.sh	Certificado autofirmado de desarrollo
migrations/17_rol_sysclin_app.sql	Rol de la aplicación con permisos mínimos

A.1 dock/docker-compose.yml

Definición de los contenedores (desarrollo).

```
services:
  db:
    image: postgres:17
    container_name: syscomcli_bd # Nombre de contenedor BD para TESIS
    restart: always
    # ESTO ACELERA LA CARGA DE DUMPS:
    command: >
      postgres  -c synchronous_commit=off  -c full_page_writes=off -c max_wal_size=3GB -c
checkpoint_timeout=20min
    # POSTGRES_USER/POSTGRES_PASSWORD/POSTGRES_DB llegan por env_file (sin ${VAR}: compose no
interpola el .env de la raiz).
    # Solo se usan al crear un volumen nuevo; con el volumen existente la clave se cambia con ALTER ROLE.
    env_file:
      - ../.env
    environment:
      - TZ=America/Asuncion # hora de Paraguay en el sistema operativo y en los logs (la base: ALTER DATABASE ...
SET timezone)
    ports:
      - "5434:5432" # Expuesto en el puerto 5434 (ya que 5432 y 5433 están en uso)
    volumes:
      - ./init-db:/docker-entrypoint-initdb.d:ro
      - tesis_syscomcli_data:/var/lib/postgresql/data
    networks:
      - network-tesis
    shm_size: '256mb'
    healthcheck:
      test: [ "CMD-SHELL", "pg_isready -U postgres" ]
      interval: 10s
      timeout: 5s
      retries: 5

  backup:
    image: postgres:17
```

```
    container_name: syscomcli_backup # Backup automatico de la BD (pg_dump programado)
    restart: always
    env_file:
      - ../.env
    environment:
      - DB_HOST=db
      - TZ=America/Asuncion # hora de Paraguay en los nombres de los backups y en el log
      - BACKUP_INTERVAL_SECONDS=86400 # cada 24h
      - BACKUP_RETENTION_DAYS=14 # se conservan los ultimos 14 dias
    entrypoint: [ "sh", "/scripts/entrypoint.sh" ]
    volumes:
      - ./backup:/scripts:ro
      - ../backups:/backups
    depends_on:
      db:
        condition: service_healthy
    networks:
      - network-tesis

  app:
    build:
      context: ..
      dockerfile: dock/Dockerfile
    container_name: syscomcli_app # Nombre de contenedor App para TESIS
    restart: always
    env_file:
      - ../.env
    environment:
      - DB_HOST=db
      - TZ=America/Asuncion # hora de Paraguay para datetime.now(), las tareas programadas y el log
    ports:
      - "5005:5000" # Flask HTTP Expuesto en el puerto 5005 (evitando 5000 y 5001)
      - "5680:5678" # debugpy (debug remoto, puerto alternativo)
    volumes:
      # Hot reload: monta la raíz del proyecto para que los cambios se reflejen al instante
      - ..:/app
    depends_on:
      db:
        condition: service_healthy
    networks:
      - network-tesis

  nginx:
    image: nginx:1.27-alpine
    container_name: syscomcli_nginx # Proxy HTTPS (termina TLS y reenvia a app:5000)
    restart: always
    # Sin env_file: nginx no necesita (ni debe ver) los secretos del .env
    environment:
      - TZ=America/Asuncion # hora de Paraguay en el log de accesos
    ports:
      - "80:80" # HTTP: redirige a HTTPS (y valida Let's Encrypt en el servidor)
      - "443:443" # HTTPS: entrada oficial al sistema
    volumes:
      - ./nginx/conf.d:/etc/nginx/conf.d:ro
      - ./nginx/certs:/etc/nginx/certs:ro
      - ./nginx/acme:/var/www/acme:ro
    depends_on:
      - app
    networks:
      - network-tesis

volumes:
  tesis_syscomcli_data: # Volumen independiente

networks:
  network-tesis: # Red independiente
    driver: bridge
```

A.2 dock/docker-compose.servidor.yml

Ajustes del servidor: puertos cerrados y HSTS.

```
# Ajustes SOLO para el servidor de despliegue (se suma a docker-compose.yml):
#   docker compose -f dock/docker-compose.yml -f dock/docker-compose.servidor.yml up -d
#
# - Los puertos de desarrollo (app 5005, BD 5434, debugpy 5680) quedan atados a 127.0.0.1:
#   desde internet solo se llega por nginx (80/443). No se depende de ufw: Docker lo saltea en Linux.
# - HSTS activado (en local romperia http://localhost:5005).
# - En el .env del servidor: SESSION_COOKIE_SECURE=1 y FLASK_DEBUG=0.

services:
  db:
    ports: !override
      - "127.0.0.1:5434:5432" # Administracion por tunel SSH

  app:
    ports: !override
      - "127.0.0.1:5005:5000"
      - "127.0.0.1:5680:5678"

  nginx:
    volumes:
      - ./nginx/hsts.conf:/etc/nginx/snippets/hsts.conf:ro
```

A.3 dock/Dockerfile

Imagen de la aplicación y modo de arranque.

```
# Usar una imagen de Python ligera
FROM python:3.12-slim

# Evitar que Python genere archivos .pyc y habilitar unbuffered logging
ENV PYTHONDONTWRITEBYTECODE=1
ENV PYTHONUNBUFFERED=1

# Instalar dependencias del sistema necesarias para psycopg2, weasyprint y reportlab
RUN apt-get update && apt-get install -y --no-install-recommends \
    build-essential \
    libpq-dev \
    shared-mime-info \
    libcairo2 \
    libpango-1.0-0 \
    libpangocairo-1.0-0 \
    libgdk-pixbuf-2.0-0 \
    libffi-dev \
    libxml2-dev \
    libxslt1-dev \
    zlib1g-dev \
    gettext \
    # Manuales: el PDF se exporta del Word estándar con LibreOffice (app/services/manual_word_service.py).
    # fonts-liberation trae Liberation Serif, que ocupa lo mismo que Times New Roman letra por letra.
    libreoffice-writer-nogui \
    python3-uno \
    fonts-liberation \
    && apt-get clean && rm -rf /var/lib/apt/lists/*

# Crear y establecer el directorio de trabajo
WORKDIR /app

# Instalar dependencias de Python
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
RUN pip install --no-cache-dir debugpy
```

```
# Copiar el resto del código
COPY . .

# Exponer los puertos de Flask y Debugpy
EXPOSE 5000 5678

# Arranque segun FLASK_DEBUG (.env):
#   1 (desarrollo): debugpy + servidor de desarrollo de Flask con recarga automatica.
#   0 (servidor):   gunicorn. UN solo worker (con hilos): el scheduler de run.py arranca
#                   por proceso, y con mas workers se duplicarian los recordatorios por WhatsApp.
CMD ["sh", "-c", "if [ \"$FLASK_DEBUG\" = \"1\" ]; then exec python -m debugpy --listen 0.0.0.0:5678 run.py; else
exec gunicorn --bind 0.0.0.0:5000 --workers 1 --threads 8 --timeout 120 --access-logfile - --error-logfile - run:app; fi"]
```

A.4 .env.example

Plantilla del .env, sin valores.

```
# Base de Datos
DB_HOST=127.0.0.1
DB_PORT=5432
DB_NAME=clinicain
# Rol de la aplicacion (permisos minimos, ver migrations/17_rol_sysclin_app.sql)
DB_USER=sysclin_app
DB_PASSWORD=
# Administracion (superusuario): contenedor db en instalacion nueva, backup y migraciones
POSTGRES_USER=postgres
POSTGRES_PASSWORD=
POSTGRES_DB=clinicain

# Flask
FLASK_APP=run.py
FLASK_ENV=development
FLASK_DEBUG=1
# Obligatoria, minimo 32 caracteres: python -c "import secrets; print(secrets.token_hex(32))"
SECRET_KEY=
# 1 solo con HTTPS (servidor); 0 en local para usar el puerto 5005 por HTTP
SESSION_COOKIE_SECURE=0

# WhatsApp Notifications (Ultramsg) - opcional
ULTRAMSG_INSTANCE_ID=
ULTRAMSG_TOKEN=

# Correo saliente (MFA, recordatorios, facturas) - opcional
SMTP_HOST=
SMTP_PORT=587
SMTP_USER=
SMTP_PASSWORD=
SMTP_FROM=
```

A.5 dock/backup/entrypoint.sh

Servicio de backup: ciclo cada 24 h.

```
#!/bin/sh

INTERVAL_SECONDS="${BACKUP_INTERVAL_SECONDS:-86400}"

echo "[$(date -Iseconds)] Servicio de backup iniciado. Intervalo: ${INTERVAL_SECONDS}s, retencion: $
{BACKUP_RETENTION_DAYS:-14} dias."

# Esperar a que la BD acepte conexiones (tras reiniciar Docker puede tardar mas que el healthcheck)
until pg_isready -h "${DB_HOST}" -p "${DB_PORT}" -U "${POSTGRES_USER}" -d "${DB_NAME}" -q; do
  echo "[$(date -Iseconds)] Esperando a la BD..."
  sleep 3
```

```
done

# Backup inmediato al arrancar; un fallo no debe matar el loop
sh /scripts/backup.sh || echo "[$(date -Iseconds)] Backup inicial fallo, se reintenta en el proximo intervalo."

while true; do
  sleep "${INTERVAL_SECONDS}"
  sh /scripts/backup.sh || echo "[$(date -Iseconds)] Backup fallo, se reintenta en el proximo intervalo."
done
```

A.6 dock/backup/backup.sh

Backup cifrado con GPG y rotación.

```
#!/bin/sh
set -e

BACKUP_DIR="/backups"
RETENTION_DAYS="${BACKUP_RETENTION_DAYS:-14}"
PUBLIC_KEY="/scripts/keys/backup_pub.asc"
TIMESTAMP=$(date +%Y%m%d_%H%M%S)
FILENAME="${DB_NAME}_${TIMESTAMP}.dump.gpg"
TMPFILE="${BACKUP_DIR}/.${FILENAME}.tmp"
# El dump en claro vive solo dentro del contenedor (/tmp), nunca en el volumen de backups
PLAINFILE="/tmp/${DB_NAME}_${TIMESTAMP}.dump"
export GNUPGHOME="/tmp/gnupg"

# Ante cualquier salida (ok o error) no quedan restos en claro ni archivos a medio escribir
trap 'rm -f "${PLAINFILE}" "${TMPFILE}"' EXIT

echo "[$(date -Iseconds)] Iniciando backup cifrado de ${DB_NAME}..."

if [ ! -f "${PUBLIC_KEY}" ]; then
  echo "[$(date -Iseconds)] ERROR: falta la clave publica ${PUBLIC_KEY}, no se crea backup sin cifrar."
  exit 1
fi

# Credenciales de administracion (POSTGRES_*): DB_USER es el rol de la app (sysclin_app), con permisos
minimos
if ! PGPASSWORD="${POSTGRES_PASSWORD}" pg_dump \
  -h "${DB_HOST}" \
  -p "${DB_PORT}" \
  -U "${POSTGRES_USER}" \
  -d "${DB_NAME}" \
  -Fc \
  -f "${PLAINFILE}"; then
  echo "[$(date -Iseconds)] ERROR: pg_dump fallo, no se creo backup."
  exit 1
fi

# Cifrado asimetrico: con la clave publica se cifra; solo la privada (fuera del servidor) descifra
mkdir -p -m 700 "${GNUPGHOME}"
if ! gpg --batch --yes --quiet --trust-model always \
  --recipient-file "${PUBLIC_KEY}" \
  --output "${TMPFILE}" \
  --encrypt "${PLAINFILE}"; then
  echo "[$(date -Iseconds)] ERROR: el cifrado fallo, no se creo backup."
  exit 1
fi

mv "${TMPFILE}" "${BACKUP_DIR}/${FILENAME}"
echo "[$(date -Iseconds)] Backup creado: ${FILENAME} ($(du -h "${BACKUP_DIR}/${FILENAME}" | cut -f1))"

echo "[$(date -Iseconds)] Rotando backups con mas de ${RETENTION_DAYS} dias..."
find "${BACKUP_DIR}" -name "${DB_NAME}_*.dump.gpg" -type f -mtime "+${RETENTION_DAYS}" -print -delete

echo "[$(date -Iseconds)] Backup finalizado. Backups actuales:"
```

```
ls -lh "${BACKUP_DIR}"/*.dump.gpg 2>/dev/null || echo "(ninguno)"
```

A.7 dock/backup/restore_test.sh

Prueba de restauración.

```
#!/bin/sh
# Prueba de restauracion de un backup cifrado (PDF 8 - Seguridad).
# Descifra el backup con la clave PRIVADA, lo restaura en una base temporal,
# compara filas de tablas clave contra la base real y borra la base temporal.
#
# Se ejecuta a mano desde la raiz del repo (pide la contrasena de la clave privada):
#   docker run --rm -it --network dock_network-tesis --env-file .env \
#     -v "$PWD/backups:/backups:ro" -v "$PWD/dock/backup:/scripts:ro" \
#     -v "<ruta>/backup_privada.asc:/clave/backup_privada.asc:ro" \
#     postgres:17 sh /scripts/restore_test.sh [nombre_del_backup.dump.gpg]
# Sin argumento usa el backup mas reciente.

BACKUP_DIR="/backups"
PRIVATE_KEY="/clave/backup_privada.asc"
TEST_DB="sysclin_restore_test"
TABLAS="usuarios pacientes citas consultas facturas auditoria_sistema"
PLAINFILE="/tmp/restore_test.dump"
export GNUPGHOME="/tmp/gnupg"
export PGHOST="${DB_HOST}" PGPORT="${DB_PORT}" PGUSER="${POSTGRES_USER}" PGPASSWORD="$
{POSTGRES_PASSWORD}"

if [ -n "$1" ]; then
  ARCHIVO="${BACKUP_DIR}/$1"
else
  ARCHIVO=$(ls -1t "${BACKUP_DIR}"/"${DB_NAME}"_*.dump.gpg 2>/dev/null | head -1)
fi

fallo() {
  echo "RESTAURACION FALLO: $1"
  exit 1
}

# Al salir (ok o error): se borran el dump en claro, la clave importada y la base temporal
trap 'rm -f "${PLAINFILE}"; rm -rf "${GNUPGHOME}"; psql -d postgres -qc "DROP DATABASE IF EXISTS $
{TEST_DB}" >/dev/null 2>&1' EXIT

[ -f "${ARCHIVO}" ] || fallo "no se encontro el backup (${ARCHIVO:-ninguno})"
[ -f "${PRIVATE_KEY}" ] || fallo "falta la clave privada montada en ${PRIVATE_KEY}"

echo "Backup a probar: $(basename "${ARCHIVO}")"

mkdir -p -m 700 "${GNUPGHOME}"
gpg --batch --quiet --import "${PRIVATE_KEY}" 2>/dev/null || fallo "no se pudo importar la clave privada"

echo "Descifrando (se pide la contrasena de la clave privada)..."
gpg --quiet --pinentry-mode loopback --output "${PLAINFILE}" --decrypt "${ARCHIVO}" || fallo "no se pudo descifrar"

psql -d postgres -qc "DROP DATABASE IF EXISTS ${TEST_DB}" >/dev/null 2>&1
psql -d postgres -qc "CREATE DATABASE ${TEST_DB}" || fallo "no se pudo crear la base temporal"

echo "Restaurando en ${TEST_DB}..."
pg_restore --exit-on-error -d "${TEST_DB}" "${PLAINFILE}" || fallo "pg_restore devolvio error"

echo ""
printf "%-20s %12s %12s\n" "tabla" "restaurada" "actual"
for t in ${TABLAS}; do
  restaurada=$(psql -d "${TEST_DB}" -Atc "SELECT count(*) FROM ${t}") || fallo "falta la tabla ${t} en el backup"
  actual=$(psql -d "${DB_NAME}" -Atc "SELECT count(*) FROM ${t}")
  printf "%-20s %12s %12s\n" "${t}" "${restaurada}" "${actual}"
done
echo "(diferencias = actividad posterior al backup)"
```

```
[ "$(psql -d "${TEST_DB}" -Atc "SELECT count(*) FROM usuarios")" -gt 0 ] || fallo "usuarios vacia en el backup"

echo ""
echo "RESTAURACION OK"
```

A.8 dock/nginx/conf.d/default.conf

Proxy HTTPS (nginx).

```
# Proxy HTTPS de Sysclin (PDF 8 - Seguridad).
# nginx termina TLS y reenvia a la app por la red interna (app:5000).
# Mismo archivo en desarrollo y en el servidor: solo cambian los certificados
# (autofirmado en local, Let's Encrypt en el servidor) y el HSTS (snippets/, solo servidor).

server_tokens off;

# HTTP: solo la validacion de Let's Encrypt; todo lo demas se redirige a HTTPS
server {
    listen 80;
    server_name _;

    location /.well-known/acme-challenge/ {
        root /var/www/acme;
    }

    location / {
        return 301 https://$host$request_uri;
    }
}

server {
    listen 443 ssl;
    http2 on;
    server_name _;

    ssl_certificate     /etc/nginx/certs/fullchain.pem;
    ssl_certificate_key /etc/nginx/certs/privkey.pem;
    ssl_protocols       TLSv1.2 TLSv1.3;
    ssl_prefer_server_ciphers off;
    ssl_session_cache   shared:SSL:10m;
    ssl_session_timeout 1d;
    ssl_session_tickets off;

    client_max_body_size 5m;

    # Encabezados de seguridad
    add_header X-Frame-Options "SAMEORIGIN" always;
    add_header X-Content-Type-Options "nosniff" always;
    add_header Referrer-Policy "strict-origin-when-cross-origin" always;

    # HSTS solo en el servidor (se monta desde docker-compose.servidor.yml).
    # En local romperia http://localhost:5005: el navegador aplica HSTS por host, no por puerto.
    include /etc/nginx/snippets/*.conf;

    location / {
        proxy_pass http://app:5000;
        # $http_host conserva el puerto: Flask-WTF compara el Referer con el Host en HTTPS (CSRF)
        proxy_set_header Host              $http_host;
        # Se sobrescribe (no se concatena) lo que mande el cliente: la IP no se puede falsificar por aca
        proxy_set_header X-Forwarded-For   $remote_addr;
        proxy_set_header X-Forwarded-Proto $scheme;
        proxy_read_timeout 120s;
    }
}
```

A.9 dock/nginx/hsts.conf

HSTS (solo servidor).

```
# HSTS: el navegador solo usara HTTPS con este dominio durante 1 año.
# Se monta SOLO en el servidor (docker-compose.servidor.yml), nunca en local.
add_header Strict-Transport-Security "max-age=31536000" always;
```

A.10 dock/nginx/generar_cert_local.sh

Certificado autofirmado de desarrollo.

```
#!/bin/sh
# Certificado autofirmado para HTTPS en desarrollo (localhost). En el servidor se usa Let's Encrypt.
# Se ejecuta desde la raiz del repo (no requiere openssl en Windows):
#   docker run --rm -v "${PWD}\dock\nginx:/nginx" postgres:17 sh /nginx/generar_cert_local.sh
set -e

CERTS_DIR="/nginx/certs"
mkdir -p "${CERTS_DIR}"

openssl req -x509 -nodes -newkey rsa:2048 -days 825 \
  -keyout "${CERTS_DIR}/privkey.pem" \
  -out "${CERTS_DIR}/fullchain.pem" \
  -subj "/CN=localhost/O=Sysclin (desarrollo)" \
  -addext "subjectAltName=DNS:localhost,IP:127.0.0.1"

echo "Certificado autofirmado creado en dock/nginx/certs/ (valido 825 dias):"
openssl x509 -in "${CERTS_DIR}/fullchain.pem" -noout -subject -enddate
```

A.11 migrations/17_rol_sysclin_app.sql

Rol de la aplicación con permisos mínimos.

```
-- 17_rol_sysclin_app.sql - Rol de la aplicacion y sus permisos
-- (antes migracion 103; PDF 8, Seguridad). Se escribe a mano, no lo genera generar_esquema_base.py.
-- Sin acentos a proposito: se puede aplicar con Get-Content de PowerShell 5.1 sin romper nada.
-- Rol de aplicacion SIN superusuario y con permisos minimos (solo DML).
-- La app (DB_USER en .env) se conecta con sysclin_app; postgres queda solo para
-- administracion, migraciones y backup (POSTGRES_USER/POSTGRES_PASSWORD en .env).
--
-- La contrasena NO se escribe en este archivo (se versiona en git): se pasa como variable de psql.
--   docker cp migrations\17_rol_sysclin_app.sql syscomcli_bd:/tmp/
--   docker exec syscomcli_bd psql -U postgres -d <base> -v ON_ERROR_STOP=1 -v
app_password='<DB_PASSWORD del .env>' -f /tmp/17_rol_sysclin_app.sql
-- OJO: sysclin_app es un rol de TODO el servidor PostgreSQL. Si en el servidor hay otra base del sistema,
-- usar la misma contrasena del .env (si no, esa base deja de poder conectarse).
-- Idempotente: se puede volver a ejecutar (por ejemplo, para rotar la contrasena).

\if :{?app_password}
\else
  \echo 'ERROR: falta -v app_password=... (ver cabecera del archivo)'
  \quit
\endif

-- 1. Rol de login sin privilegios de administracion
SELECT 'CREATE ROLE sysclin_app LOGIN NOSUPERUSER NOCREATEDB NOCREATEROLE
NOREPLICATION NOBYPASSRLS'
WHERE NOT EXISTS (SELECT 1 FROM pg_roles WHERE rolname = 'sysclin_app') \gexec

ALTER ROLE sysclin_app PASSWORD :'app_password';

COMMENT ON ROLE sysclin_app IS
```

```
    'Rol de la aplicacion Sysclin: solo SELECT/INSERT/UPDATE/DELETE, uso de secuencias y ejecucion de
funciones. Sin DDL, TRUNCATE ni superusuario. La administracion usa postgres.';

-- 2. Conexion a la base: solo la app (postgres es superusuario y no necesita GRANT)
SELECT format('REVOKE CONNECT, TEMPORARY ON DATABASE %I FROM PUBLIC', current_database())
\gexec
SELECT format('GRANT CONNECT ON DATABASE %I TO sysclin_app', current_database()) \gexec

-- 3. Permisos minimos sobre los objetos existentes (sin CREATE, TRUNCATE, REFERENCES ni TRIGGER)
GRANT USAGE ON SCHEMA public TO sysclin_app;
GRANT SELECT, INSERT, UPDATE, DELETE ON ALL TABLES IN SCHEMA public TO sysclin_app;
GRANT USAGE, SELECT ON ALL SEQUENCES IN SCHEMA public TO sysclin_app;
GRANT EXECUTE ON ALL ROUTINES IN SCHEMA public TO sysclin_app;

-- 4. Mismos permisos para lo que postgres cree despues (vistas, procedimientos, auditoria: PDF 4, 5, 9)
ALTER DEFAULT PRIVILEGES FOR ROLE postgres IN SCHEMA public
    GRANT SELECT, INSERT, UPDATE, DELETE ON TABLES TO sysclin_app;
ALTER DEFAULT PRIVILEGES FOR ROLE postgres IN SCHEMA public
    GRANT USAGE, SELECT ON SEQUENCES TO sysclin_app;
ALTER DEFAULT PRIVILEGES FOR ROLE postgres IN SCHEMA public
    GRANT EXECUTE ON ROUTINES TO sysclin_app;

-- 5. Hora de Paraguay en la base (PDF 9 + 6, paso 1b). pg_dump no guarda esta configuracion, por eso va aca.
--    Sin esto la base trabaja en UTC y now() / CURRENT_TIMESTAMP guardan la hora 3 h adelantada.
--    Vale para todas las conexiones nuevas a esta base (sysclin_app y postgres).
SELECT format('ALTER DATABASE %I SET timezone = %L', current_database(), 'America/Asuncion') \gexec

-- 6. Auditoria inmutable (PDF 9 + 6, paso 2). Va despues del punto 3 porque ese da UPDATE/DELETE en todas las
tablas.
--    auditoria_transacciones: la app solo LEE; la escribe el trigger fn_auditar_cambios (SECURITY DEFINER).
--    auditoria_sistema: la app inserta eventos, pero no puede modificarlos ni borrarlos.
REVOKE INSERT, UPDATE, DELETE, TRUNCATE ON auditoria_transacciones FROM sysclin_app;
REVOKE UPDATE, DELETE, TRUNCATE ON auditoria_sistema FROM sysclin_app;
REVOKE ALL ON FUNCTION fn_auditar_cambios() FROM PUBLIC, sysclin_app;
```